

第 04 讲

Matrix Multiplication: 为什么神经网络几乎到处都在做矩阵乘法

本讲目标：从“每一个输出格子都是行向量与列向量的 Dot Product”出发，真正理解矩阵乘法的 Shape 规则、手算过程、神经网络中的 XW ，并把它连接到 Transformer 的 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ 。

讲义完整版

目录

1 为什么这一讲是深度学习的核心数学关口	3
1.1 本讲学完后的最低验收标准	3
2 矩阵乘法到底解决什么问题?	4
2.1 【原理】把大量 Dot Product 一次算完	4
2.2 【通俗解释】“一排问题”去问“一组模板”	4
3 先记住 Shape 规则: $(m, n) @ (n, p) \rightarrow (m, p)$	4
3.1 为什么必须“中间维相等”?	5
4 【具体数字例子】2×3 乘 3×2, 完整手算	5
4.1 第一个输出: C_{11}	5
4.2 第二个输出: C_{12}	6
4.3 第三、第四个输出	6
5 每一个输出格子究竟代表什么?	6
5.1 【公式逐项拆解】	6
6 Matrix Multiplication 与 Element-wise Multiplication 不一样	7
7 为什么神经网络用矩阵乘法? 从一个神经元开始	7
8 从一个样本到一个 Batch: XW	8
9 Linear Transformation: 矩阵乘法不只是“算数”	8
10 Transformer 中的 $Q = XW_Q$ 到底发生了什么?	9
11 矩阵乘法的顺序不能随便交换	10
12 转置为什么会频繁和矩阵乘法一起出现?	10
13 【代码实验】NumPy 中怎样做矩阵乘法	11
13.1 为什么本讲还要手写 manual_matmul?	11
14 常见误区	12
14.1 误区 1: 只会背 Shape 口诀, 不知道为什么	12
14.2 误区 2: 认为输出 Shape 由元素总数决定	12

14.3 误区 3: $A * B$ 就是矩阵乘法	12
14.4 误区 4: W 只是 “一堆随机数字”	12
14.5 误区 5: 矩阵乘法只是为了提速	12
15 与后续课程的关系	12
16 本讲实验任务	13
17 本讲总结	13

1 为什么这一讲是深度学习的核心数学关口

前三讲我们已经建立了三块基础：Tensor 是怎样组织数字的、Shape 怎样描述各个轴、两个向量怎样通过 Dot Product 得到一个标量分数。

现在进入一个真正的分水岭：**怎样一次完成大量 Dot Product?**

答案就是 Matrix Multiplication（矩阵乘法）。神经网络里几乎所有核心计算都可以追溯到矩阵乘法：

- 全连接层 / Linear Layer: $Y = XW + b$;
- MLP: 连续做多次矩阵乘法再加非线性;
- Embedding 后的线性投影;
- Transformer: $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$;
- Attention score: QK^T ;
- 最终 vocabulary projection: hidden state 乘词表输出矩阵。

本讲最重要的一句话

矩阵乘法 $C = AB$ 的每一个输出元素 C_{ij} ，本质上都是： **A 的第 i 行，与 B 的第 j 列做一次 Dot Product。**

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 课程里已经见过 $Q = XW_Q$ 。当时可以把它理解成“把 X 映射成 Query”。本讲要真正解释： X 里面的一行是什么、 W_Q 的一列是什么、为什么相乘以后 Shape 会变，以及每一个 Query 数字到底是怎样算出来的。

1.1 本讲学完后的最低验收标准

你应该能做到：

1. 看到 $(m,n)@(n,p)$ ，立即判断输出 Shape 是 (m,p) ;
2. 解释为什么“中间两个维度必须相等”;
3. 手算一个 2×3 乘 3×2 的矩阵;
4. 解释一个输出格子为什么是“行 dot 列”;
5. 区分矩阵乘法与逐元素乘法;
6. 看懂一批样本如何用 XW 一次经过多个神经元;
7. 看懂 $Q = XW_Q$ 的 Shape 变化;
8. 遇到矩阵乘法 Shape 报错时，知道先检查哪两个维度。

2 矩阵乘法到底解决什么问题？

2.1 【原理】把大量 Dot Product 一次算完

上一讲中，一个 Query q 和一个 Key k 做点积，会得到一个标量。如果现在有 100 个 Query 和 100 个 Key，当然可以写很多循环逐个计算，但这种方式既难读又慢。

矩阵乘法的价值在于：把许多向量整齐地堆成矩阵，再用一条规则一次完成大量配对计算。

2.2 【通俗解释】“一排问题”去问“一组模板”

假设一名学生有 3 个特征：

$$x = [\text{数学}, \text{英语}, \text{编程}]$$

现在我们想根据这 3 个特征同时计算 2 个结果：

- 科研潜力分数；
- 工程实践分数。

那么每个输出都需要一组权重。把两组权重放到一起，就得到一个矩阵。一个输入向量与这个权重矩阵相乘，就能同时算出两个结果。

为什么不是一个神经元一个神经元地算？

理论上可以。工程上不这样做，因为现代硬件最擅长大块矩阵运算。把几千、几万个神经元的计算整理成一次矩阵乘法，才是深度学习能高效运行的重要原因之一。

3 先记住 Shape 规则： $(m, n) @ (n, p) \rightarrow (m, p)$

设：

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p}$$

则：

$$C = AB \in \mathbb{R}^{m \times p}$$

Shape 口诀

$$(m, n) @ (n, p) \rightarrow (m, p)$$

中间维必须相等，外侧维留下来。

例如：

$$(2, 3) @ (3, 4) \rightarrow (2, 4)$$

因为第一个矩阵有 3 列，第二个矩阵有 3 行，所以可以相乘。

但是：

$$(2, 3) @ (2, 4)$$

不能直接相乘，因为左边的列数是 3，右边的行数是 2，3 与 2 对不上。

3.1 为什么必须“中间维相等”？

因为每个输出格子都要做一次 Dot Product。Dot Product 要求两个向量长度相同。

A 的一行长度是 n ； B 的一列长度也是 n 。如果两个长度不同，就没有办法一一对应相乘。

最常见的初学者错误

不要用“两个矩阵 Shape 一样”来判断能不能做矩阵乘法。矩阵乘法并不要求两个 Shape 相同，只要求左矩阵的最后一个参与维度与右矩阵的第一个参与维度匹配。二维情况下就是“左列数 = 右行数”。

4 【具体数字例子】 2×3 乘 3×2 ，完整手算

设：

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

先看 Shape：

$$A : (2, 3), \quad B : (3, 2)$$

因此：

$$C = AB : (2, 2)$$

输出 C 有 2 行 2 列，一共有 4 个数字。我们逐个算。

4.1 第一个输出： C_{11}

取 A 第一行：

$$[1, 2, 3]$$

取 B 第一列：

$$\begin{bmatrix} 7 \\ 9 \\ 11 \end{bmatrix}$$

做点积：

$$C_{11} = 1 \times 7 + 2 \times 9 + 3 \times 11 = 7 + 18 + 33 = 58$$

4.2 第二个输出： C_{12}

还是取 A 第一行，但这次取 B 第二列：

$$C_{12} = 1 \times 8 + 2 \times 10 + 3 \times 12 = 8 + 20 + 36 = 64$$

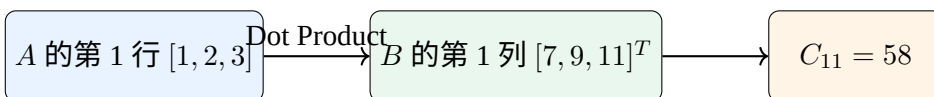
4.3 第三、第四个输出

$$C_{21} = 4 \times 7 + 5 \times 9 + 6 \times 11 = 139$$

$$C_{22} = 4 \times 8 + 5 \times 10 + 6 \times 12 = 154$$

所以：

$$AB = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$



真正需要形成的心智模型

不要把矩阵乘法看成“一个特殊公式”。把它看成一个 **Dot Product 工厂**：左矩阵提供很多“行向量”，右矩阵提供很多“列向量”，矩阵乘法把每个行-列组合都计算一次。

5 每一个输出格子究竟代表什么？

一般地：

$$C = AB$$

其中：

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

5.1 【公式逐项拆解】

- i ：选择 A 的第几行；
- j ：选择 B 的第几列；
- k ：沿着两个长度相同的向量逐项配对；
- A_{ik} ：第 i 行第 k 个数；
- B_{kj} ：第 j 列第 k 个数；
- $\sum$ ：把所有逐项乘积加起来。

现在不需要背下标公式

对零基础来说，最重要的不是背 $C_{ij} = \sum_k A_{ik} B_{kj}$ ，而是看到公式时能翻译成一句话：**输出第 i 行第 j 列 = 左边第 i 行 dot 右边第 j 列。**

6 Matrix Multiplication 与 Element-wise Multiplication 不一样

设：

$$M_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad M_2 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

逐元素乘法：

$$M_1 \odot M_2 = \begin{bmatrix} 1 \times 5 & 2 \times 6 \\ 3 \times 7 & 4 \times 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

矩阵乘法：

$$M_1 M_2 = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

在 NumPy 中：

```
M1 * M2    # 逐元素乘法
M1 @ M2    # 矩阵乘法
```

常见误区 / 风险

这两个运算恰好都能作用于两个 2×2 矩阵，所以最容易混淆。以后看到代码中的 $*$ 与 $@$ ，要立即意识到它们是完全不同的运算。

7 为什么神经网络用矩阵乘法？从一个神经元开始

假设一个样本有 3 个输入特征：

$$x = [x_1, x_2, x_3]$$

一个神经元有 3 个权重：

$$w = [w_1, w_2, w_3]$$

忽略 Bias 时，神经元首先计算：

$$z = x \cdot w$$

也就是：

$$z = x_1 w_1 + x_2 w_2 + x_3 w_3$$

如果不是 1 个神经元，而是 4 个神经元呢？每个神经元都有一套自己的权重。把 4 套权重作为 4 列

放进矩阵 W ：

$$W \in \mathbb{R}^{3 \times 4}$$

那么：

$$xW \in \mathbb{R}^{1 \times 4}$$

一次就得到 4 个神经元的输出。

为什么 W 的一列对应一个输出单元？

因为 xW 的每个输出元素，都是输入向量 x 与 W 的某一列做 Dot Product。每一列就相当于某个输出单元的一整套权重。

8 从一个样本到一个 Batch: XW

现实训练不会只处理一个样本。假设一个 Batch 有 32 个样本，每个样本 768 个特征：

$$X.shape = (32, 768)$$

现在要映射成 3072 个输出特征：

$$W.shape = (768, 3072)$$

因此：

$$Y = XW$$

Shape 为：

$$(32, 768) @ (768, 3072) \rightarrow (32, 3072)$$

这就是 Transformer FFN 中常见的大矩阵计算

例如 hidden size = 4096，MLP intermediate size = 11008，那么会出现类似：

$$(B \times T, 4096) @ (4096, 11008)$$

或者保持三维 Batch/Sequence 结构，由框架在最后一维上做等价的线性变换。底层本质仍然是“每个 Token 表示乘同一个权重矩阵”。

9 Linear Transformation：矩阵乘法不只是“算数”

矩阵 W 可以看作一个变换规则：把一个向量从一种表示空间映射到另一种表示空间。

如果：

$$x \in \mathbb{R}^3, \quad W \in \mathbb{R}^{3 \times 2}$$

则：

$$y = xW \in \mathbb{R}^2$$

可以理解为：把一个 3 维表示压成一个 2 维表示。

如果：

$$W \in \mathbb{R}^{3 \times 8}$$

则是把 3 维输入映射成 8 维输出。

科研里为什么经常看到 projection / projection layer?

Projection（投影/映射）在深度学习论文里通常就是一个可学习的线性变换。作者改变 projection dimension, 本质上是在改变“输入特征如何被重新组合, 以及输出表示空间有多宽”。后续的 Ablation 可能会比较不同 hidden dimension、bottleneck dimension 或 projection head。

10 Transformer 中的 $Q = XW_Q$ 到底发生了什么?

设一个极小序列有 4 个 Token, 每个 Token 的 hidden size = 3:

$$X.shape = (4, 3)$$

为了得到 2 维 Query:

$$W_Q.shape = (3, 2)$$

则:

$$Q = XW_Q$$

Shape:

$$(4, 3) @ (3, 2) \rightarrow (4, 2)$$

这句话逐层翻译就是:

1. X 有 4 行, 每一行对应一个 Token 的当前表示;
2. 每行有 3 个数字, 即 hidden dimension = 3;
3. W_Q 有 2 列, 相当于定义 2 个新的输出特征;
4. 每个 Token 的 3 维表示分别和 W_Q 两列做点积;
5. 每个 Token 得到 2 个新数字, 所以最终是 $(4, 2)$ 。

同理:

$$K = XW_K, \quad V = XW_V$$

为什么同一个 X 会变成不同的 $Q/K/V$?

因为 W_Q 、 W_K 、 W_V 是三套不同的可学习参数。输入完全一样, 但通过不同矩阵重新组合后, 会产生三种不同的表示, 用于不同角色: Query 用来“发问”, Key 用来“被匹配”, Value 用来“提供信息”。

11 矩阵乘法的顺序不能随便交换

一般情况下：

$$AB \neq BA$$

甚至可能 AB 能算，而 BA 根本不能算。

例如：

$$A : (2, 3), \quad B : (3, 4)$$

则：

$$AB : (2, 4)$$

但：

$$BA : (3, 4) @ (2, 3)$$

因为 4 与 2 不匹配，所以无法相乘。

不要把矩阵乘法当普通数字乘法

标量有 $ab = ba$ ，所以初学者容易下意识认为矩阵也能交换。矩阵表示“变换”，而连续变换的先后顺序通常不同，因此矩阵乘法一般不满足交换律。

12 转置为什么会频繁和矩阵乘法一起出现？

矩阵转置会交换行和列。例如：

$$K.shape = (T, D)$$

转置后：

$$K^T.shape = (D, T)$$

因此：

$$QK^T : (T, D) @ (D, T) \rightarrow (T, T)$$

这正是 Attention 的关键 Shape：每个 Query Token 都要和每个 Key Token 计算一个分数，所以会得到一个 $T \times T$ 的 score matrix。

提前看一眼 Attention score matrix

如果序列长度 $T = 4$ ，那么 QK^T 是 4×4 ：第 i 行第 j 列的数字，就是“第 i 个 Token 的 Query 与第 j 个 Token 的 Key”的点积分数。

这就是为什么第 02 讲学 transpose、第 03 讲学 dot product、第 04 讲学 matrix multiplication：三块知识现在开始合流。

13 【代码实验】NumPy 中怎样做矩阵乘法

最小代码：

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=float)

B = np.array([[7, 8],
              [9, 10],
              [11, 12]], dtype=float)

C = A @ B
print(A.shape) # (2, 3)
print(B.shape) # (3, 2)
print(C.shape) # (2, 2)
print(C)
```

输出：

```
[[ 58.  64.]
 [139. 154.]]
```

13.1 为什么本讲还要手写 `manual_matmul`?

因为如果一开始只写：

```
C = A @ B
```

你知道“能运行”，却未必知道内部逻辑。本讲实验会先用三层小循环自己实现：

```
for i in range(rows):
    for j in range(cols):
        for k in range(inner):
            C[i, j] += A[i, k] * B[k, j]
```

再和 NumPy 结果核对。

核心概念

以后进入 PyTorch 后，`torch.matmul`、`@`、`nn.Linear` 会把底层细节封装起来。但你现在先手写一遍，是为了建立“框架到底替我做了什么”的最低闭环能力。

14 常见误区

14.1 误区 1：只会背 Shape 口诀，不知道为什么

“中间消掉、外面留下”只是结果。原因是：输出格子来自行 dot 列，所以这两个向量必须等长。

14.2 误区 2：认为输出 Shape 由元素总数决定

矩阵乘法与 reshape 不同。reshape 要保持元素总数，矩阵乘法会通过乘加运算创造新的数值，输出元素数量完全可以改变。

14.3 误区 3： $A * B$ 就是矩阵乘法

NumPy/PyTorch 中， $*$ 通常是逐元素乘法， $@$ 才是矩阵乘法。

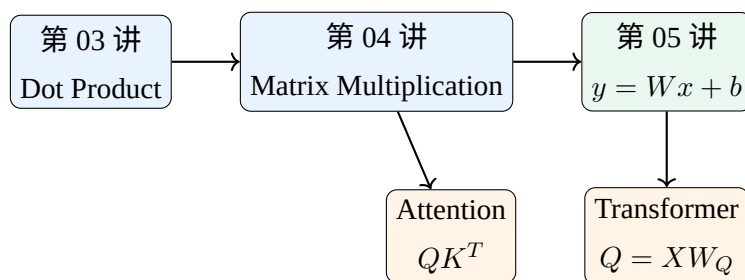
14.4 误区 4： W 只是“一堆随机数字”

初始化时可能随机，但训练后 W 是模型通过梯度学习出来的参数。它决定输入特征如何被组合成新的表示。

14.5 误区 5：矩阵乘法只是为了提速

不仅是提速。矩阵乘法本身就是线性层定义的一部分，它表达了“一个输入如何通过一套可学习权重映射到另一个特征空间”。

15 与后续课程的关系



下一讲将在矩阵乘法上正式加入两个概念：**Weight（权重）**与**Bias（偏置）**，并彻底拆开：

$$y = Wx + b$$

为什么它构成一个 Linear Layer，以及每个数字到底代表什么。

16 本讲实验任务

完整实验文件：lesson04_matrix_multiplication_experiment.py。

实验包含：

1. 手算 2×3 与 3×2 矩阵乘法；
2. 自己写 manual_matmul 与 NumPy 对照；
3. 预测 $(4, 3) @ (3, 5)$ 的 Shape；
4. 故意制造 Shape 不匹配并阅读报错；
5. 比较逐元素乘法与矩阵乘法；
6. 用 XW 模拟一批样本通过线性权重；
7. 用 XW_Q 、 XW_K 、 XW_V 模拟微型 Transformer；
8. 可选使用 PyTorch 再核对一次。

科研训练：从现在开始养成“先写 Shape”习惯

以后阅读论文或调试模型代码时，不要只看函数名。建议在草稿纸上写：

$$X : (B, T, D), \quad W_Q : (D, d_k), \quad Q : (B, T, d_k)$$

很多所谓“复杂模型错误”，最后只是某个轴理解错了。能追 Shape，是读懂深度学习代码和模型结构图的最低基本功之一。

17 本讲总结

必须留下来的 6 条结论

1. 矩阵乘法就是大量 Dot Product 的结构化计算；
2. $(m, n) @ (n, p) \rightarrow (m, p)$ ；
3. 中间维必须相等，因为 Dot Product 的两个向量必须等长；
4. 输出 C_{ij} = 左矩阵第 i 行 dot 右矩阵第 j 列；
5. XW 可以一次让一个 Batch 的所有样本经过同一套权重；
6. Transformer 的 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ 本质上就是同一条规则。

本节检查点

如果现在给你：

$$X : (32, 128, 768), \quad W : (768, 3072)$$

你暂时不需要掌握三维 Tensor 的所有矩阵乘法细节，但应该立刻看出：最后一个 768 与权重矩阵的 768 匹配，结果的最后一维会变成 3072。后面学习 PyTorch Linear 时会系统解释三维输入如何处理。